

Sumario

1. Visao Geral	2
1.1 Proposito	2
1.2 Publico-Alvo e Casos de Uso	2
1.3 Principios de Design	2
2. Arquitetura do Sistema	3
2.1 Visao Geral da Arquitetura	3
2.2 Componentes Principais	3
2.3 Stack Tecnologica	4
3. Motor de Navegacao (Navigator)	5
3.1 Gerenciamento do Browser	5
3.2 Configuracao Anti-Deteccao	5
4. Estrategias de Raspagem	6
4.1 Classificacao de Sites	6
4.2 Extracao de Dados Estruturados	6
4.3 Sistema de Seletores Inteligentes	7
5. Sistema de Paginacao	7
5.1 Crawl recursivo com controle de escopo	8
6. Pipeline de Limpeza e Validacao	8
6.1 Limpeza de Dados	8
6.2 Validacao com Schema (Zod)	9
7. Sistema de Exportacao	9
8. Rate Limiting e Conformidade Etica	10
8.1 Controle de Frequencia	10
8.2 Respeito a robots.txt e sitemaps	10
8.3 Conformidade LGPD / GDPR	10
9. Sistema de Tratamento de Erros e Recuperacao	11
10. Interface de Configuracao	12
11. Monitoramento e Observabilidade	12
12. Exemplos de Uso	13
12.1 Exemplo Basico - Raspagem de uma pagina	13
12.2 Exemplo Avancado - Crawl com paginacao	13
12.3 Exemplo com API Interception	14
13. Estrutura do Projeto	14
14. Roadmap de Evolucao	15

1. Visao Geral

1.1 Proposito

Este documento de design (DESIGN.md) especifica a arquitetura, os componentes e os fluxos de uma skill de raspagem web (web scraping) capaz de extrair dados de qualquer site de forma ponta a ponta, desde a navegacao e renderizacao da pagina ate a extracao, limpeza e exportacao dos dados. A skill foi projetada para funcionar como um modulo autonomo e reutilizavel, integravel em pipelines de automacao, sistemas de inteligencia artificial e workflows de processamento de dados. O design prioriza flexibilidade, robustez e conformidade etica, garantindo que a raspagem possa ser realizada de forma confiavel em diferentes tipos de sites, desde paginas estaticas simples ate aplicacoes web complexas com renderizacao do lado do cliente (SPA), protecoes anti-bot e conteudo dinâmico.

A arquitetura proposta utiliza um motor de navegacao baseado em Playwright (headless browser) como camada principal de interacao com os sites, complementado por um sistema inteligente de seletores CSS/XPath que se adaptam automaticamente a mudancas na estrutura do HTML. O pipeline de processamento e dividido em fases claramente definidas: descoberta de URLs, navegacao e renderizacao, selecao e extracao de dados, limpeza e normalizacao, e exportacao em multiplos formatos. Cada fase e isolada e configuravel independentemente, permitindo adaptacao a cenarios especificos sem modificacao do core.

1.2 Publico-Alvo e Casos de Uso

A skill foi projetada para atender diferentes perfis de usuarios tecnicos, desde engenheiros de dados que precisam coletar informacoes estruturadas da web para analise, ate equipes de growth que monitoram concorrentes e precos, e desenvolvedores que integram dados externos em suas aplicacoes. Os casos de uso principais incluem: monitoramento de precos e disponibilidade de produtos em e-commerces, coleta de dados de imoveis para plataformas de real estate, agregacao de noticias e artigos de fontes diversas, extracao de dados financeiros de portais de investimento, catalogacao de perfis profissionais em redes sociais, compilacao de dados academicos de repositórios de pesquisas, e alimentacao de LLMs com dados atualizados extraídos automaticamente da web.

1.3 Principios de Design

O design da skill segue sete principios fundamentais que guiam todas as decisoes arquiteturais:

- **Modularidade:** Cada componente do pipeline (navigacao, extracao, limpeza, exportacao) e independente e pode ser substituido ou estendido sem afetar os demais. A arquitetura baseada em plugins permite adicionar novos extratores, formatos de saida e estrategias de navegacao.
- **Resiliencia:** O sistema deve lidar graciosamente com falhas de rede, timeouts, mudancas no layout do site, blocos anti-bot e erros inesperados, implementando retry automatico com backoff exponencial e fallback para estrategias alternativas de extracao.

- **Transparencia:** Todas as acoes de raspagem devem ser registradas em logs detalhados, permitindo auditoria, debug e conformidade regulatoria. O usuario deve ter visibilidade completa sobre o que foi extraido, de onde e quando.
- **Eficiencia:** A raspagem deve minimizar o consumo de recursos (CPU, memoria, banda) e tempo de execucao, utilizando tecnicas como concorrencia controlada, cache de paginas, e seletores otimizados que evitam processamento desnecessario.
- **Etica:** A skill deve respeitar robots.txt, limites de rate, termos de servico dos sites e leis de protecao de dados (LGPD/GDPR). O usuario deve poder configurar o nivel de agressividade da raspagem.
- **Configurabilidade:** Todos os parametros criticos (timeout, user-agent, proxy, rate limit, seletor) devem ser expostos como opcoes de configuracao, permitindo adaptacao sem modificacao de codigo.
- **Simplicidade de Uso:** A API deve ser intuitiva, com funcoes de alto nivel que abstraem a complexidade da raspagem, permitindo que um usuario extraia dados de uma pagina com poucas linhas de codigo.

2. Arquitetura do Sistema

2.1 Visao Geral da Arquitetura

A arquitetura da skill segue o padrao de pipeline em fases, onde cada fase recebe a saida da fase anterior e produz dados para a proxima. O fluxo principal e: Configuracao -> Navegacao -> Renderizacao -> Extracao -> Limpeza -> Validacao -> Exportacao. Cada fase e encapsulada em um modulo independente com interface bem definida, permitindo composicao flexivel e teste unitario isolado. O pipeline pode ser executado de forma sequencial (uma pagina por vez) ou em batch (multiplas paginas em paralelo), dependendo da configuracao do usuario e dos limites do site alvo.

O componente central e o ScrapeEngine, que orquestra a execucao do pipeline. Ele recebe uma configuracao (ScrapeConfig), gerencia o ciclo de vida do navegador, coordena as fases e coleta os resultados. O motor suporta dois modos de operacao: single-page (extracao de uma unica pagina) e crawl (navegacao recursiva a partir de uma URL semente, seguindo links internos respeitando regras de escopo). O design favorece composicao sobre heranca, utilizando interfaces e injecao de dependencias para permitir extensibilidade sem acoplamento forte entre componentes.

2.2 Componentes Principais

Componente	Responsabilidade	Tecnologia	Interface
ScrapeEngine	Orquestra o pipeline completo	TypeScript	scrape(config) -> Result

Navigator	Gerencia ciclo de vida do browser	Playwright	goto(url), click(sel)
Renderer	Aguarda renderizacao completa	Playwright + JS	waitFor(selector, timeout)
Extractor	Selecione e extrai dados do DOM	Cheerio / DOM API	extract(schema, html)
Cleaner	Limpa e normaliza dados extraidos	TypeScript puro	clean(rawData) -> CleanData
Validator	Valida tipos e integridade	Zod / Joi	validate(data, schema)
Paginator	Navega paginacao de listas	Playwright	hasNext() -> next()
RateLimiter	Controla frequencia de requisicoes	Token Bucket	acquire() -> Promise
AntiDetect	Evasao de deteccao anti-bot	Stealth plugins	apply(context)
Exporter	Exporta dados em multiplos formatos	TypeScript	export(data, format)
Logger	Registro detalhado de acoes	Winston / Pino	info/warn/error(msg)
Cache	Cache de respostas HTTP e paginas	In-memory / Redis	get(key) / set(key, val)

Tabela 1: Componentes principais da skill e suas responsabilidades.

2.3 Stack Tecnologica

Camada	Tecnologia	Versao	Justificativa
Runtime	Node.js	20+ (LTS)	Ecosistema maduro para scraping e automacao web
Navegador Headless	Playwright	1.40+	Multi-browser, API moderna, auto-wait nativo
Parsing HTML	Cheerio	1.0+	jQuery-like, rapido, sem browser overhead
Validacao	Zod	3.20+	TypeScript-first, erros amigaveis, schemas compostos
HTTP Fallback	Axios + Undici	1.6+ / 6.0+	Requisicoes HTTP puras para APIs e sitemaps
Exportacao CSV	PapaParse	5.4+	Streaming, grandes arquivos, encoding flexivel
Exportacao Excel	ExcelJS	4.4+	XLSX com formatacao, multi-sheet, streaming
Log	Pino	8.0+	JSON estruturado, alto desempenho, multi-transport
Cache	node-cache / Redis	- / 7.0+	In-memory para dev, Redis para producao
Proxy	proxy-chain	2.4+	Rotacao de proxies, autenticacao, verificacao
Scheduling	node-cron	3.0+	Agendamento de tasks periodicas de scraping
Tipagem	TypeScript	5.3+	Seguranca de tipos em todo o codebase

Tabela 2: Stack tecnologica completo da skill.

3. Motor de Navegacao (Navigator)

3.1 Gerenciamento do Browser

O Navigator e o componente responsavel por gerenciar o ciclo de vida completo do navegador headless. Ele encapsula a criacao, configuracao e destruicao de instancias do browser Playwright, garantindo que os recursos sejam liberados corretamente apos o uso. O design suporta tres modos de operacao: headless (sem interface grafica, modo padrao para producao), headed (com interface grafica visivel, util para debug) e remote (conectado a um browser remoto via CDP, para escalabilidade horizontal). O gerenciamento inclui configuracao de contexto de navegacao (viewports, user-agents, geolocation, locale, timezone), interceptacao de requisicoes de rede (para bloqueio de recursos desnecessarios como imagens, fontes e stylesheets, acelerando o carregamento), e injecao de scripts customizados no contexto da pagina antes do carregamento do HTML.

Cada instancia do browser opera em um contexto isolado (BrowserContext), o que permite executar multiplas sessoes de raspagem em paralelo dentro do mesmo processo, cada uma com seus proprios cookies, storage e configuracoes. O Navigator implementa um pool de contextos reutilizaveis, reduzindo o overhead de criacao de novos contextos para cada pagina. Quando uma sessao de scraping e finalizada, o contexto e resetado (cookies limpos, storage limpo) e retornado ao pool, em vez de ser destruido. Essa estrategia reduz significativamente o tempo total de execucao em operacoes de crawl com centenas ou milhares de paginas.

3.2 Configuracao Anti-Deteccao

O modulo AntiDetect aplica uma camada de ofuscacao sobre o navegador headless para evitar deteccao por scripts anti-bot. As tecnicas implementadas incluem: remocao de propriedades reveladoras do objeto navigator (webdriver, languages, plugins), injecao de WebGL e Canvas fingerprint realistas, emulacao de interacoes humanas (movimentos de mouse com curvas de Bezier, delays aleatorios entre teclas, scroll natural com aceleracao e desaceleracao), spoofing de headers HTTP (User-Agent rotativo, Accept-Language, Sec- headers), e manipulacao do viewport e media queries para simular um ambiente de desktop ou mobile real. O modulo tambem suporta integracao com servicos de fingerprint rotation como AdsPower ou Multilogin para cenarios que exigem nivel maximo de evasao.

Tecnica	Protecao	Implementacao	Impacto
User-Agent rotativo	Fingerprinting basico	Pool de 500+ UAs atualizados	Alto
WebDriver removal	navigator.webdriver	Object.defineProperty override	Critico
Canvas fingerprint	Canvas/Audio fingerprint	Injecao de noise sutil	Medio

WebGL spoofing	WebGL fingerprint	Vendor/Renderer override	Medio
Mouse emulation	Comportamento automatizado	Bezier curves + jitter	Alto
Header spoofing	Header analysis	Sec-Fetch, Accept, Lang	Critico
Viewport emulation	Device detection	Real device profiles	Alto
Cookie isolation	Sessao tracking	Contexto isolado por sessao	Critico
JS challenge solver	Cloudflare, Akamai	Delay adaptativo + espera	Alto

Tabela 3: Tecnicas anti-deteccao e niveis de impacto.

4. Estrategias de Raspagem

4.1 Classificacao de Sites

A skill identifica automaticamente o tipo de site alvo e seleciona a estrategia de raspagem mais adequada. A classificacao e baseada em analise heuristica do HTML inicial, headers HTTP e comportamento de JavaScript. O sistema distingue cinco categorias principais de sites, cada uma exigindo abordagens diferentes de renderizacao e extracao. A classificacao correta e fundamental para escolher o modo de renderizacao (HTTP puro vs. browser completo), o timing de espera apropriado, e as tecnicas de extracao mais eficazes para cada tipo de pagina.

Categoria	Caracteristicas	Estrategia	Exemplos
Estatico	HTML completo no response, sem JS	HTTP puro + Cheerio	Blogs WordPress, Wikis, Gov
SPA (React/Vue/Angular)	HTML vazio, conteudo via JS	Playwright + waitFor	Gmail, Twitter, Airbnb
Hybrido	HTML base + enriquecimento JS	HTTP + Playwright fallback	Amazon, LinkedIn, Medium
Protegido (anti-bot)	Cloudflare, Akamai, Captcha	Stealth + delay + solver	Bot-protection sites
API-driven	Dados via XHR/fetch em JSON	Interceptacao de rede	Apps com API publica

Tabela 4: Classificacao de sites e estrategias recomendadas.

4.2 Extracao de Dados Estruturados

O Extrator é o componente central de coleta de dados, responsável por mapear elementos do DOM para campos de dados estruturados. O usuário define um schema de extração declarativo que especifica, para cada campo desejado, o seletor CSS ou XPath, o tipo de dado esperado (texto, atributo, HTML interno, link, imagem, preço, data), e transformações opcionais (regex, trim, parse de data, conversão numérica). O schema funciona como um contrato entre a intenção do usuário e a estrutura da página, permitindo que mudanças no layout do site sejam absorvidas pela atualização dos seletores sem modificação do pipeline.

O extrator suporta extração de dados de três tipos de fontes: elementos individuais (um seletor retorna um valor), listas (um seletor pai retorna N itens filhos com sub-seletores), e tabelas (extração automática de dados tabulares com detecção de headers). Para cada tipo, o sistema aplica automaticamente normalização de espaços em branco, remoção de caracteres invisíveis (zero-width spaces, BOM), e decodificação de entidades HTML. O resultado é um array de objetos JSON com campos tipados e validados, pronto para consumo por etapas posteriores do pipeline ou exportação direta.

4.3 Sistema de Seletores Inteligentes

Além dos seletores CSS e XPath manuais, a skill implementa um sistema de seletores inteligentes que ajudam a lidar com mudanças de layout do site alvo. Os seletores inteligentes utilizam múltiplas estratégias de fallback encadeadas: se o seletor primário falhar, o sistema tenta automaticamente seletores alternativos baseados em heurísticas como atributos data-test, aria-label, texto do elemento, posição relacional e proximidade semântica. O usuário pode definir um array de seletores candidatos para cada campo, ordenados por prioridade, e o sistema escolherá o primeiro que retornar resultados.

Tipo de Seletor	Prioridade	Exemplo	Robustez
data-test / data-testid	1 (maior)	[data-test="price"]	Muito alta
aria-label	2	[aria-label="Product name"]	Alta
ID estavel	3	#product-price	Alta
Classe semantica	4	.product-card .price	Media
Estrutura relativa	5	.card > div:nth-child(2)	Baixa
Texto do elemento	6	:contains("Add to cart")	Baixa
XPath complexo	7 (fallback)	//div[@class="item"]//span	Media

Tabela 5: Tipos de seletores ordenados por robustez contra mudanças.

5. Sistema de Paginação

O Paginador é o componente responsável por detectar e navegar automaticamente sistemas de paginação em listas de resultados. Ele analisa a estrutura da página para identificar padrões comuns de paginação:

botoes "Proximo" / "Anterior", links numericos de pagina, scroll infinito, e botes de "Carregar mais". A deteccao e baseada em heurísticas que combinam seletores CSS com analise de texto (busca por termos como "next", "proxima", "more", "carregar mais" em diferentes idiomas). O sistema suporta cinco padroes de paginacao, cada um com logica de navegacao dedicada e estrategias de espera especificas para garantir que todos os itens sejam carregados antes da extracao.

Padrao	Deteccao	Navegacao	Limite
Paginacao numerada	Botoes/a com numeros sequenciais	Clique no proximo numero	Definido pelo user
Next/Prev	Botoes "Proximo" / "Anterior"	Clique no "Proximo"	Ate desabilitar
Scroll infinito	Ausencia de paginacao visivel	Scroll ate final + wait	Max pages config
Load more button	Botao "Carregar mais"	Clique + espera conteudo	Ate botao sumir
URL params	?page=2, ?offset=20	Incrementa param na URL	Ate 404/vazio

Tabela 6: Padroes de paginacao suportados com estrategias de navegacao.

5.1 Crawl recursivo com controle de escopo

Para operacoes de crawl que requerem navegacao alem de paginas de listas, o sistema implementa um crawler recursivo com controle de escopo rigoroso. O crawler recebe uma URL semente e um conjunto de regras de escopo que definem quais links podem ser seguidos. As regras incluem: prefixo de URL permitido (ex.: apenas links dentro de /products/), profundidade maxima (ex.: no maximo 3 niveis de profundidade a partir da semente), filtros por padrao de URL (regex de inclusao/exclusao), e limites de dominio (ex.: apenas links do mesmo dominio ou subdominios especificos). O crawler mantem uma fila de URLs a visitar, um conjunto de URLs ja visitadas (para evitar ciclos) e uma frontier queue com prioridade baseada na relevancia do link. O politeness (delay entre requisicoes ao mesmo dominio) e configuravel e respeita automaticamente as diretrizes do robots.txt.

6. Pipeline de Limpeza e Validacao

6.1 Limpeza de Dados

O Cleaner recebe os dados brutos extraídos pelo Extrator e aplica uma sequencia de transformacoes para normaliza-los em um formato consistente e utilizavel. As operacoes de limpeza sao definidas por tipo de dado e incluem: normalizacao de texto (remocao de espacos extras, tabs, newlines duplicados, caracteres zero-width, entidades HTML, tags HTML residuais), normalizacao de numeros (remocao de simbolos de moeda, separadores de milhar, conversao para float/int), normalizacao de datas (parser flexivel que aceita formatos DD/MM/YYYY, MM/DD/YYYY, ISO 8601, texto relativo como "ha 3 dias"),

normalizacao de URLs (resolucao de paths relativos para absolutos, remocao de tracking params como `utm_source`, normalizacao de `www/http/https`), e deduplicacao de registros baseada em hash dos campos primarios. Cada transformacao e idempotente, garantindo que execucoes repetidas nao alterem o resultado.

Tipo de Dado	Transformacoes	Exemplo de Entrada	Saida
Texto	Trim, collapse whitespace, strip HTML	" Produto X "	"Produto X"
Preco	Remove R\$, sep. milhar, to float	"R\$ 1.299,90"	1299.90
Data	Parse flexivel para ISO 8601	"15 de Mar, 2026"	"2026-03-15"
URL	Resolve relativa, remove tracking	"/p?id=5&utm;=x"	"https://site.com/p?id=5"
Telefone	Strip non-digits, add country code	" +55 (11) 99999-0000"	" +5511999990000"
Bool	Parse sim/nao/true/false/1/0	"Sim"	true

Tabela 7: Transformacoes de limpeza por tipo de dado com exemplos.

6.2 Validacao com Schema (Zod)

Apos a limpeza, os dados passam por um processo de validacao baseado em schemas Zod que garante integridade tipologica e a presenca de campos obrigatorios. O schema de validacao e derivado automaticamente do schema de extracao definido pelo usuario, com regras adicionais de consistencia. Se um registro falhar na validacao, ele e marcado com status "invalid" e incluído no relatorio de erros (junto com o motivo da falha), mas nao e descartado silenciosamente. O validador tambem detecta anomalias estatisticas, como valores muitos acima ou abaixo da media, e sinaliza esses casos para revisao manual. O resultado da validacao e um objeto estruturado contendo o array de registros validos, o array de registros invalidos com erros, e estatisticas de qualidade (taxa de validacao, campos mais frequentemente ausentes, distribuicao de erros por campo).

7. Sistema de Exportacao

O Exporter suporta saida em multiplos formatos, permitindo que os dados extraídos sejam consumidos por diferentes sistemas e ferramentas. Cada formato possui configuracoes especificas como encoding, delimitador, formatacao de numeros/datas, e compressao. O exportador e projetado para lidar com datasets grandes de forma eficiente, utilizando streaming quando possivel para evitar carregar todo o dataset em memoria. O usuario pode configurar multiplos formatos de saida simultaneos, e o sistema

gera todos eles em uma unica execucao do pipeline.

Formato	Biblioteca	Max Recomendado	Caracteristicas
JSON	fs.writeFileSync	100MB+	Estruturado, aninhado, ideal para APIs
CSV	PapaParse unparse	500K+ linhas	Universal, Excel-compatible, streaming
XLSX (Excel)	ExcelJS	1M+ celulas	Multi-sheet, formatacao, formulas
HTML	Template strings	Ilimitado	Visual, web-ready, customizavel
XML	xml2js	50MB+	Estruturado, schema-validavel
Markdown	Custom formatter	Ilimitado	Tabelas, listas, legivel
Parquet	apache-arrow	Big data	Colunar, compressao, analytics-ready

Tabela 8: Formatos de exportacao suportados com limites e caracteristicas.

8. Rate Limiting e Conformidade Etica

8.1 Controle de Frequencia

O RateLimiter implementa o algoritmo Token Bucket para controlar a frequencia de requisicoes ao site alvo. O algoritmo permite rajadas curtas de requisicoes (burst) dentro de um limite maximo, mas mantem uma media ao longo do tempo que respeita o rate configurado. O usuario pode configurar tres parametros: requests per second (RPS), max burst size, e per-domain delay. O sistema tambem suporta adaptive rate limiting, onde o RPS e automaticamente reduzido quando o site alvo retorna codigos de status 429 (Too Many Requests) ou 503 (Service Unavailable), e gradualmente restaurado apos o site voltar a responder normalmente. Essa estrategia auto-reguladora protege tanto o site alvo (evitando sobrecarga) quanto o scraper (evitando bans por excesso de requisicoes).

8.2 Respeito a robots.txt e sitemaps

A skill analisa automaticamente o robots.txt do dominio alvo antes de iniciar a raspagem, verificando quais paths sao permitidos ou bloqueados para user-agents genericos ou especificos. URLs bloqueadas por robots.txt sao excluidas automaticamente do escopo do crawl, e o usuario recebe um aviso no log. O sistema tambem pode extrair e utilizar sitemaps.xml para descoberta eficiente de URLs, priorizando as URLs listadas no sitemap em vez de seguir links do HTML (mais lento e menos completo). O sitemap e processado incrementalmente, permitindo iniciar a raspagem antes de baixar todo o sitemap, e URLs com lastmod recente sao priorizadas para maximizar a relevancia dos dados coletados.

8.3 Conformidade LGPD / GDPR

A skill inclui funcionalidades para auxiliar na conformidade com legislacoes de protecao de dados. O sistema detecta automaticamente banners de consentimento de cookies (GDPR) e pode aceita-los ou rejeita-los conforme a configuracao. Dados pessoais (emails, telefones, CPFs, enderecos) sao marcados com metadados de sensibilidade no resultado da extracao, permitindo ao usuario aplicar anonimizacao automatica ou excluir esses campos da exportacao. O log de raspagem registra a data, hora, URL e dados extraídos, proporcionando rastreabilidade para fins de auditoria. O usuario e responsavel por verificar se a raspagem dos dados esta de acordo com os termos de servico do site e a legislacao aplicavel, mas a skill fornece as ferramentas para facilitar essa conformidade.

9. Sistema de Tratamento de Erros e Recuperacao

O tratamento de erros e projetado em tres niveis hierarquicos: nivel de requisicao (falhas individuais de carregamento de pagina), nivel de extracao (falhas no parsing de elementos especificos) e nivel de pipeline (falhas estruturais que impedem a execucao). Cada nivel tem sua propria estrategia de recovery. Erros de requisicao triggeram retry com backoff exponencial (1s, 2s, 4s, 8s, maximo 3 tentativas), alternancia de User-Agent entre tentativas, e eventual fallback para HTTP puro se o browser falhar repetidamente. Erros de extracao triggeram fallback para seletores alternativos, e se todos falharem, o campo e marcado como null com registro do erro no log detalhado. Erros de pipeline causam parada graciosa com salvamento automatico dos dados coletados ate o momento do erro.

Tipo de Erro	Codigo	Acao Automatica	Max Retries
Timeout de navegacao	ERR_TIMEOUT	Retry + aumentar timeout	3
Pagina nao encontrada	404	Log + pular URL	0
Rate limited	429	Backoff + reduzir RPS	5
Erro de servidor	5xx	Retry com backoff	3
Seletor nao encontrado	SEL_MISSING	Fallback para alternativo	0
JS nao carregou	RENDER_FAIL	Retry + espera estendida	2
Captcha detectado	CAPTCHA	Pausa + notifica usuario	0
Conexao recusada	ECONNREFUSED	Retry + trocar proxy	3
Erro de parsing	PARSE_ERR	Log + campo null	0
Memoria insuficiente	OOM	Fechar contexto + reiniciar	1

Tabela 9: Catalogo de erros com acoes automaticas de recuperacao.

10. Interface de Configuracao

Toda a configuracao da skill e centralizada em um objeto TypeScript (ScrapeConfig) que e passado como argumento para o ScrapeEngine. A configuracao utiliza o padrao "convention over configuration", onde valores padrao sao fornecidos para todos os parametros, permitindo ao usuario override apenas os parametros que precisa personalizar. A configuracao pode ser fornecida como objeto JavaScript/TypeScript inline, como arquivo JSON externo, ou como variaveis de ambiente (para deploy em containers). A validacao da configuracao e realizada em tempo de inicializacao pelo schema Zod, e erros de configuracao invalida sao reportados imediatamente com mensagens descritivas, evitando falhas em runtime.

Parametro	Tipo	Padrao	Descricao
url	string	-	URL inicial ou lista de URLs para iniciar a raspagem
mode	enum	single	single (uma pagina) ou crawl (navegacao recursiva)
browser	enum	chromium	chromium, firefox, webkit
headless	boolean	true	Executar browser em modo headless
timeout	number	30000	Timeout de navegacao em milissegundos
rateLimit	number	2	Maximo de requisicoes por segundo
maxPages	number	100	Maximo de paginas para crawl
depth	number	3	Profundidade maxima de navegacao
proxy	string	none	URL do proxy (http://user:pass@host:port)
userAgent	string	auto	User-Agent personalizado ou rotacao automatica
output	string[]	["json"]	Formatos de exportacao desejados
respectRobotsTxt	boolean	true	Respeitar diretrizes do robots.txt
antiDetect	boolean	true	Ativar tecnicas anti-deteccao

Tabela 10: Parametros de configuracao com tipos e valores padrao.

11. Monitoramento e Observabilidade

O sistema de monitoramento fornece visibilidade completa sobre a execucao da raspagem em tempo real e historico. O Logger registra todos os eventos em formato JSON estruturado com campos de timestamp, nivel (info, warn, error), modulo de origem, URL sendo processada, e dados do evento. Os

logs são escritos em dois destinos simultâneos: console (para desenvolvimento) e arquivo rotacionado (para produção). Além dos logs textuais, o sistema emite métricas quantitativas que podem ser consumidas por sistemas de monitoramento como Prometheus ou Grafana: número total de páginas processadas, taxa de sucesso/falha por URL, tempo médio de carregamento por página, quantidade de dados extraídos, utilização de memória do processo, e número de requisições bloqueadas pelo rate limiter.

O monitoramento inclui também um sistema de alertas configuráveis que disparam notificações quando determinados thresholds são atingidos: taxa de erro acima de 10% (alerta), acima de 30% (crítico), tempo médio de página acima de 5 segundos, número de páginas sem dados extraídos acima de 5 consecutivas, e memória acima de 80% do limite configurado. Os alertas podem ser enviados via webhook, email, ou integração com Slack/Discord. O dashboard de monitoramento fornece uma visão consolidada com gráficos de progresso, contadores de páginas, erros recentes e estimativa de tempo restante para a conclusão.

12. Exemplos de Uso

12.1 Exemplo Básico - Raspagem de uma página

O exemplo mais simples de uso da skill é a raspagem de dados estruturados de uma única página. O usuário define a URL, o schema de extração com seletores CSS para cada campo desejado, e o formato de saída. O motor cuida de todo o resto: navega até a URL, espera a renderização completa, extrai os dados conforme o schema, limpa e valida os resultados, e exporta no formato solicitado. O código abaixo demonstra uma extração de produtos de uma página de e-commerce, com seletores para título, preço, avaliação e link do produto.

```
import { scrape } from 'web-scraper';  
  
const result = await  
scrape({  
  url:  
    'https://exemplo.com/produtos',  
  schema:  
    [{ field: 'titulo', selector: '.product-title' },  
    { field: 'preco', selector: '.price', type: 'number' },  
    { field: 'avaliacao', selector: '.rating', type: 'number' },  
    { field: 'link', selector: 'a.product', attr: 'href' }],  
  output: ['json', 'csv'],  
  rateLimit: 2,  
});  
  
// result.data => Array de objetos extraídos  
// result.files => { json: './output/data.json', csv: './output/data.csv' }
```

12.2 Exemplo Avançado - Crawl com paginação

Para extrair dados de múltiplas páginas, o modo crawl com paginação automática é o mais indicado. O usuário configura a URL semente, o seletor da lista de itens, o padrão de paginação, e o número máximo

de paginas. O sistema automaticamente navega pelas paginas, extrai os dados de cada uma, e consolida tudo em um unico dataset. O exemplo abaixo demonstra um crawl completo de um catalogo de produtos com paginacao numerada, incluindo deteccao automatica do botao "Proximo" e parada quando o botao e desabilitado (indicando a ultima pagina).

[illegible]

12.3 Exemplo com API Interception

Muitos sites modernos carregam dados via chamadas API (XHR/fetch) em formato JSON, o que é mais eficiente e confiável do que raspar o HTML renderizado. A `skill` suporta interceptação de requisições de rede para capturar respostas de API diretamente. O usuário especifica um padrão de URL ou `content-type` para filtrar as requisições desejadas, e o sistema coleta todas as respostas correspondentes durante a navegação, eliminando a necessidade de seletores CSS e tornando a extração imune a mudanças de layout.

[illegible]

13. Estrutura do Projeto

Caminho	Tipo	Descricao
src/engine/	Modulo	Orquestrador principal do pipeline de raspagem
src/navigator/	Modulo	Gerenciamento do browser Playwright

src/extractor/	Modulo	Motor de seletores e extracao de dados do DOM
src/cleaner/	Modulo	Transformacoes e normalizacao de dados
src/validator/	Modulo	Validacao de schemas Zod e verificacao de tipos
src/paginator/	Modulo	Deteccao e navegacao automatica de paginacao
src/anti-detect/	Modulo	Tecnicas de evasao anti-bot e fingerprint spoofing
src/exporter/	Modulo	Exportacao em JSON, CSV, XLSX, HTML, XML, Parquet
src/rate-limiter/	Modulo	Controle de frequencia com Token Bucket
src/logger/	Modulo	Logging estruturado com Pino
src/cache/	Modulo	Cache in-memory e Redis para respostas HTTP
src/types/	Modulo	Interfaces TypeScript e tipos compartilhados
src/config/	Modulo	Schema Zod de validacao da configuracao
src/utils/	Modulo	Funcoes utilitarias (URL, regex, encoding)
tests/	Dir	Testes unitarios e de integracao (Vitest)
examples/	Dir	Exemplos de uso com diferentes cenarios

Tabela 11: Estrutura de diretorios do projeto com modulos e descricoes.

14. Roadmap de Evolucao

Fase	Feature	Prioridade	Complexidade
Fase 1	Motor de navegacao Playwright com anti-detect	Critica	Alta
Fase 1	Extracao por seletores CSS/XPath com fallback	Critica	Media
Fase 1	Exportacao JSON e CSV	Critica	Baixa
Fase 1	Rate limiting com Token Bucket	Critica	Baixa
Fase 2	Paginacao automatica (5 padroes)	Alta	Media
Fase 2	Pipeline de limpeza e validacao Zod	Alta	Media
Fase 2	Crawl recursivo com escopo configuravel	Alta	Alta
Fase 2	Exportacao XLSX e HTML	Alta	Media
Fase 3	Interceptacao de API (XHR/fetch)	Media	Alta
Fase 3	Dashboard de monitoramento em tempo real	Media	Alta

Fase 3	Suporte a proxy rotativo	Media	Baixa
Fase 3	Agendamento com node-cron	Media	Baixa
Fase 4	CapSolver / 2Captcha integration	Baixa	Media
Fase 4	Exportacao Parquet para Big Data	Baixa	Media
Fase 4	Plugin system para extensoes customizadas	Baixa	Alta

Tabela 12: Roadmap de evolucao por fase, prioridade e complexidade.